

fontana_escritorio

Versión de escritorio (Python + Tkinter) de fontana_movimientos

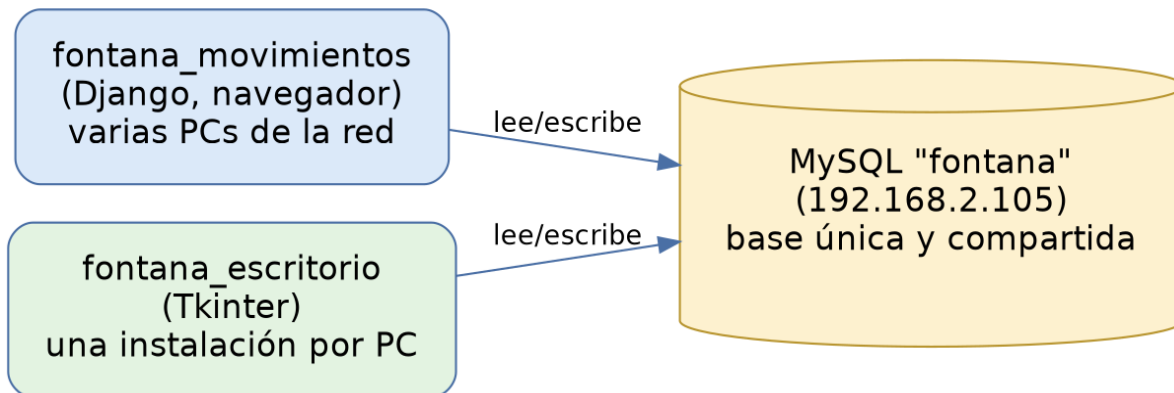
Documento de referencia — 15 de septiembre de 2026

1. Qué es y por qué existe

fontana_movimientos es un sistema grande: 13 apps de Django (entidades, productos, movimientos, movimientos_caja, comprobantes, retenciones, retenciones_inym, liquidaciones, respaldo, tipos, solicitudes_compra, empleados, remitos), con MySQL como base y acceso por navegador desde varias PCs de la red.

fontana_escritorio es la versión de ese mismo sistema como aplicación de escritorio, hecha en Python con Tkinter. No es una copia ni un sistema aparte: se conecta DIRECTO a la misma base MySQL de producción, como un cliente más -- los mismos datos, en tiempo real, en paralelo con la web.

Migrar los 13 módulos de una sola vez no es realista en una sola sesión de trabajo, así que el proyecto se arma módulo por módulo, reusando siempre las mismas tablas y las mismas reglas de negocio que ya tiene la versión Django, para no duplicar lógica que después se desincroniza entre las dos versiones. Hasta ahora se avanzó con Entidades, Productos y Movimientos (los tres completos) y Comprobantes (cabecera completa, renglones pendientes).



Ambas versiones (web y escritorio) conviven y comparten la misma base.

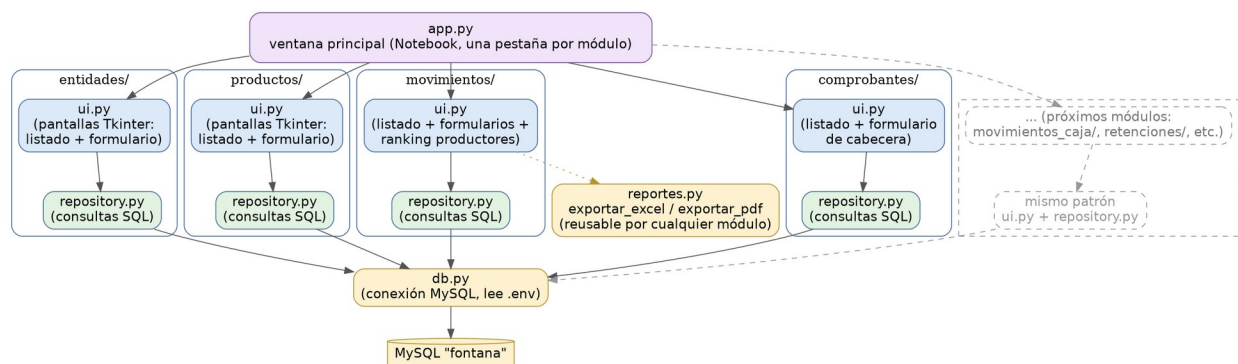
2. Arquitectura interna

Cada módulo sigue siempre el mismo patrón de dos archivos, para que el acceso a datos se pueda revisar o probar por separado de la interfaz gráfica:

- **repository.py:** las consultas SQL (listar, obtener, crear, actualizar, dar de baja/eliminar). No conoce nada de Tkinter.
- **ui.py:** las pantallas (listado con Treeview + formularios en ventanas emergentes). Llama al repository correspondiente y muestra los resultados o errores.

A partir de Movimientos se sumó también un módulo compartido, reusable por cualquier pantalla que necesite exportar un listado:

- **reportes.py:** exportar_excel(ruta, resultado) y exportar_pdf(ruta, titulo, resultado) -- adaptación a escritorio (guarda en un archivo local en vez de descargar desde el navegador) de services/reportes.py del proyecto Django, con el mismo formato de tabla y estilos.



app.py arma la ventana principal (una pestaña por módulo); db.py centraliza la conexión a MySQL, leyendo las credenciales de un archivo .env.

3. Decisiones de diseño

- **Sin login propio:** por ahora esta app no pide usuario/contraseña (Django sí lo tiene). Si hace falta restringir el acceso igual que en la web, se puede sumar más adelante.
- **IDs de tablas legadas asignados a mano:** "entidad", "producto_detalle" y "comprobante" no se tratan como autoincrementales -- se calcula $\text{MAX}(\text{id})+1$ antes de cada alta, igual que hace el propio proyecto Django para estas mismas tablas, para no arriesgarse a chocar con un id ya usado si la columna no fuera realmente AUTO_INCREMENT.
- **"movimiento" es la excepción:** a diferencia de las anteriores, el modelo Django la declara como AutoField real, así que en Movimientos el alta deja que MySQL asigne el id (`cursor.lastrowid`) en vez de calcularlo a mano.
- **Baja de Entidad = activo=0, nunca DELETE:** mismo criterio que la web -- no se borra una entidad, se la marca inactiva y se puede reactivar.
- **Eliminar Producto sí es DELETE real,** pero con un chequeo previo de uso (movimientos, renglones de comprobante, renglones de remito) y, además, si la base rechaza el borrado por una relación real, se captura ese error y se avisa en vez de que la app se caiga. El chequeo de uso es una réplica hecha desde afuera del código Django real (`productos.views.producto_eliminar`); conviene compararlo contra esa vista si se quiere blindar del todo.
- **Eliminar Movimiento** intenta el DELETE directo y muestra un aviso si la base lo rechaza (por ejemplo por tener pesaje o datos de H.V. de Yerba Mate asociados) -- mismo resultado que ve el usuario en la web, sin tener que adivinar acá todas las tablas que podrían referenciarlo.
- **Eliminar Comprobante** replica a mano el efecto de `on_delete=CASCADE` que tiene Django: se borran primero sus renglones (`comprobante_renglon / comprobante_renglon_detalle`) y su tipo de cambio si tenía, y recién ahí la cabecera. El chequeo específico de la vista Django (no dejar borrar un comprobante ya incluido en una Liquidación) todavía NO está replicado -- Liquidaciones sigue pendiente de investigar.

4. Estado actual

Módulo	Estado	Notas
Entidades	✓ Hecho	Listado con búsqueda y filtro de inactivas, alta, edición, dar de baja/reactivar.
Productos	✓ Hecho	Listado con búsqueda, alta, edición, eliminar con chequeo de uso.
Movimientos	✓ Hecho	Listado/búsqueda, alta, edición, eliminar; recepción y salida canchada de H.V. Yerba Mate; salida genérica; ranking de productores con exportación a Excel/PDF.
Comprobantes	● Parcial	Cabecera completa (listado, alta, edición, eliminar). Faltan renglones, tipo de cambio y reportes/rankings.

Movimientos de Caja	☐ Pendiente	Próximo módulo a implementar.
Retenciones / Retenciones INYM	☐ Pendiente	
Liquidaciones	☐ Pendiente	
Remitos	☐ Pendiente	
Cuenta Corriente de Productos	☐ Pendiente	Ya está terminado del lado Django; falta portarlo acá.
Solicitudes de Compra	☐ Pendiente	
Empleados	☐ Pendiente	
Tipos	☐ Pendiente	
Respaldo	☐ A evaluar	Es un módulo de backup del servidor; puede no tener sentido igual en un cliente de escritorio.

5. Módulos terminados / en curso

5.1 Entidades

- Listado con búsqueda por nombre o CUIT, y checkbox para mostrar también las inactivas.
- Alta y edición con todos los campos: nombre, CUIT, dirección, documento N°, código, localidad, código postal, IVA, provincia.
- Dar de baja / reactivar (nunca elimina, ver decisiones de diseño).
- Doble clic en una fila abre la edición directamente.

5.2 Productos

- Listado con búsqueda por nombre, mostrando también el tipo de ítem (Producto, Servicio, etc.).
- Alta y edición: nombre + tipo (combo desplegable con los item_tipo existentes).
- Eliminar, bloqueado si el producto está en uso.

5.3 Movimientos

- Listado/búsqueda con los mismos 4 filtros que la web: receptor, ID, fecha y N°.
- Alta y edición genérica: fecha, N°, entidad emisora, entidad receptora, total, unidad de medida y producto.
- Recepción de H.V. de Yerba Mate: producto y unidad fijos, receptor fijo (Fontana Secadero), se elige el operador INYM emisor, se cargan bruto/tara/descuento y el total se calcula solo; N°/fecha se precargan con el último + 1.
- Salida de Yerba Mate Canchada: mismo mecanismo al revés (emisor fijo Fontana Secadero, se elige el operador INYM receptor).
- Salida genérica: un movimiento de salida simple (fecha, producto, entidad receptora, N°, total), sin operadores INYM ni pesaje; el emisor es siempre Fontana y el total se carga directo.
- Ranking de Productores: agrupa por entidad emisora sumando el total entregado y contando entregas, con % de participación, filtro de rango de fecha y de producto, y exportación a Excel y PDF.
- Eliminar: intenta el DELETE directo y avisa si la base lo rechaza por integridad (ver decisiones de diseño).

5.4 Comprobantes (cabecera)

- Listado/búsqueda por entidad, ID y fecha, más el checkbox "Sin renglones cargados"; muestra también la cantidad de renglones de cada uno.
- Alta y edición de los 17 campos de la cabecera: entidad emisora, tipo de comprobante, tipo de documento, fecha, punto de venta, N°, moneda, neto gravado, neto no gravado, recargo, impuesto, IVA, exento, otros tributos, total, detalle y "es emisor" (rol respecto de Fontana).
- Eliminar: borra a mano los renglones y el tipo de cambio asociados antes que la cabecera, para replicar el mismo efecto en cascada que tiene Django (ver decisiones de diseño).
- Pendiente para una próxima vuelta: renglones de comprobante (cantidad/precio/IVA/unidad), tipo de cambio para moneda extranjera, y los reportes/rankings/exportaciones propios de este módulo.

6. Instalación y uso

Requisitos: Python 3 con Tkinter (incluido en la instalación estándar de Windows) y las dependencias del archivo requirements.txt (PyMySQL, python-dotenv, openpyxl y reportlab -- estas dos últimas se sumaron para la exportación a Excel/PDF de Movimientos).

```
cd fontana_escritorio
pip install -r requirements.txt
copy .env.example .env
(completar .env con los mismos datos que fontana_movimientos\.env)
python app.py
```

El archivo .env no se genera automáticamente a partir del de Django por seguridad (para no manipular la contraseña real desde afuera) -- alcanza con copiar el archivo existente a esta carpeta.

7. Cómo se probó

Antes de entregar el código de Entidades y Productos se corrió una prueba automática con una pantalla virtual (Xvfb): se abrió la ventana principal, se simularon altas, ediciones, bajas/reactivaciones y el bloqueo de "eliminar producto en uso" con datos de prueba (sin tocar la base real), confirmando que ninguna de esas acciones tira error y que las pantallas se ven correctamente.

Para Movimientos y Comprobantes se siguió otro camino de verificación, sin correr la interfaz gráfica esta vez: antes de escribir cada pieza se revisó el código Django real (views.py / forms.py / models.py del app correspondiente) para copiar la misma lógica de negocio en vez de inventarla; después, cada función de repository.py (armado de SQL, cálculo de ranking, alta con id manual, borrado en cascada) se ejecutó con una conexión MySQL simulada -- sin tocar la base de producción -- para confirmar que no tira errores y devuelve lo esperado; y por último se compiló (py_compile) y se revisó la estructura (AST) de todos los archivos del proyecto. Las funciones de exportación de reportes.py también se probaron generando un Excel y un PDF de ejemplo con datos ficticios.

8. Estructura de carpetas

```
fontana_escritorio/
├─ app.py                # ventana principal
├─ db.py                 # conexión a MySQL, lee .env
├─ reportes.py           # exportar_excel / exportar_pdf
├─ requirements.txt
├─ .env.example
├─ entidades/
│   └─ repository.py
│   └─ ui.py
```

```
|— productos/
|   |— repository.py
|   |— ui.py
|— movimientos/
|   |— repository.py
|   |— ui.py
|— comprobantes/
|   |— repository.py
|   |— ui.py
```

9. Roadmap confirmado

Orden acordado para seguir agregando módulos, pensado por dependencias. Se puede reordenar en cualquier momento.

- 1. Movimientos -- hecho.
- 2. Comprobantes -- cabecera hecha; sigue con renglones. Lo usan Liquidaciones y Cuenta Corriente de Productos.
- 3. Movimientos de Caja -- próximo módulo a implementar.
- 4. Retenciones + Retenciones INYM
- 5. Liquidaciones
- 6. Remitos
- 7. Cuenta Corriente de Productos
- 8. Solicitudes de Compra
- 9. Empleados
- 10. Tipos

(Respaldo queda a evaluar aparte, ver tabla de estado.)